



山东大学

信息科学与工程学院

XXX—XXX 学年第二学期

课程实验报告

课程名称: 数据结构与算法导论

实验名称: ADT 表的编程与实现

专 业 班 级 通信二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2026 年 5 月 13 日

一、实验目的

1. 加深对抽象数据类型 ADT 栈和队列的理解

二、实验原理

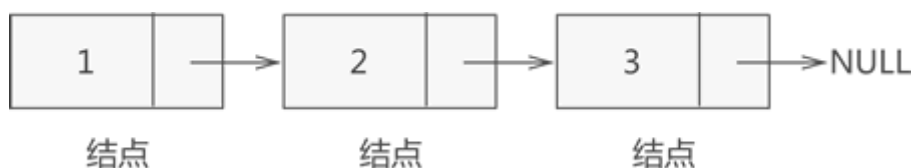
参照课本 p.32-45, 及 Figure3.6-3.13

链表, 别名链式存储结构或单链表, 用于存储逻辑关系为 "一对一" 的数据。与顺序表不同, 链表不限制数据的物理存储状态, 换句话说, 使用链表存储的数据元素, 其物理存储位置是随机的, 每个数据元素在存储时都配备一个指针, 用于指向自己的直接后继元素。

链表中每个数据的存储都由以下两部分组成: 数据元素本身, 其所在的区域称为数据域; 指向直接后继元素的指针, 所在的区域称为指针域, 在本次实验中, 我规定数据域为整型, 指针域指向下一个节点, 最后一个节点的指针域为 NULL。如下图所示:

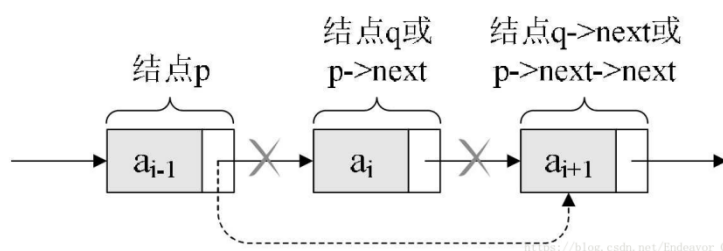


链表实际存储的是一个一个的节点, 真正的数据元素包含在这些节点中, 如下图所示:

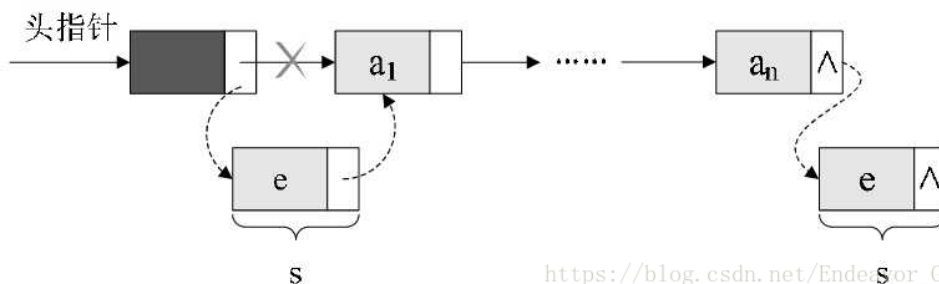


因此, 链表中每个节点的具体实现, 需要使用 C 语言中的结构体。

删除节点: 将删除节点上一节点的指针指向删除节点下一节点, 释放删除节点即可完成节点删除



插入节点: 为新入节点分配内存, 将新入节点指针指向下一节点, 再将上一节点指针指向新入节点 (顺序不能乱) 不然会造成数据丢失。



三、实验内容

编写程序实现 ADT 表的定义，及常用操作：

- 1) 判断表是否为空
- 2) 获取第 i 个节点的内容
- 3) 删除
- 4) 插入

四、实验要求

- 1) 复习 C 语言相关知识；
- 2) 实现完整的 ADT 表结构及操作，并给出应用。

五、实验源程序

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXSIZE 100

typedef char ElemType[50];

typedef struct Node {
    ElemType data;
    struct Node *next;
} Node, *LinkList;

/* 初始化线性表 */
void InitList(LinkList *L) {
    *L = (LinkList)malloc(sizeof(Node));
    if (*L == NULL) {
        printf("初始化失败，内存不足! \n");
        exit(1);
    }
    (*L)->next = NULL;
}

/* 判断线性表是否为空 */
int IsEmpty(LinkList L) {
    if (L->next == NULL) {
        return 1;
    }
    return 0;
}
```

```

}
/* 求线性表长度 */
int GetLength(LinkList L) {
    int count = 0;
    Node *p = L->next;
    while (p != NULL) {
        count++;
        p = p->next;
    }
    return count;
}

/* 取出第 i 个元素 */
int GetElem(LinkList L, int i, ElemType e) {
    int j = 1;
    Node *p = L->next;
    if (i < 1) {
        return 0;
    }
    while (p != NULL && j < i) {
        p = p->next;
        j++;
    }
    if (p == NULL) {
        return 0;
    }
    strcpy(e, p->data);
    return 1;
}

/* 在第 i 个位置插入元素 */
int InsertElem(LinkList L, int i, const char *e) {
    int j = 0;
    Node *p = L;
    Node *s = NULL;
    if (i < 1) {
        return 0;
    }
    while (p != NULL && j < i - 1) {
        p = p->next;
        j++;
    }
    if (p == NULL) {

```

```

        return 0;
    }
    s = (Node *)malloc(sizeof(Node));
    if (s == NULL) {
        printf("插入失败, 内存不足! \n");
        exit(1);
    }
    strncpy(s->data, e, 49);
    s->data[49] = '\0';
    s->next = p->next;
    p->next = s;

    return 1;
}

/* 删除第 i 个元素 */
int DeleteElem(LinkList L, int i, ElemType e) {
    int j = 0;
    Node *p = L;
    Node *q = NULL;

    if (i < 1) {
        return 0;
    }

    while (p->next != NULL && j < i - 1) {
        p = p->next;
        j++;
    }
    if (p->next == NULL) {
        return 0;
    }
    q = p->next;
    p->next = q->next;
    strcpy(e, q->data);
    free(q);
    return 1;
}

/* 输出线性表 */
void PrintList(LinkList L) {
    int i = 1;
    Node *p = L->next;

    if (IsEmpty(L)) {
        printf("\n 当前线性表为空。 \n");
    }
}

```

```

        return;
    }
    printf("\n 当前线性表内容为: \n");
    while (p != NULL) {
        printf("第 %d 个元素: %s\n", i, p->data);
        p = p->next;
        i++;
    }
}

/* 销毁线性表 */
void DestroyList(LinkList *L) {
    Node *p = NULL;
    Node *q = NULL;
    if (*L == NULL) {
        return;
    }
    p = *L;
    while (p != NULL) {
        q = p->next;
        free(p);
        p = q;
    }
    *L = NULL;
}

/* 创建初始线性表 */
void CreateList(LinkList L) {
    int n;
    int i;
    char value[50];
    printf("请输入需要建立的元素个数: ");
    scanf("%d", &n);
    if (n < 0) {
        printf("元素个数不能为负数, 已自动设置为 0. \n");
        n = 0;
    }
    for (i = 1; i <= n; i++) {
        printf("请输入第 %d 个元素的值: ", i);
        scanf("%49s", value);
        InsertElem(L, i, value);
    }
}

/* 菜单 */

```

```

void Menu(void) {

    printf("1. 判断线性表是否为空\n");
    printf("2. 求线性表长度\n");
    printf("3. 取第 i 个元素\n");
    printf("4. 在第 i 个位置插入元素\n");
    printf("5. 删除第 i 个元素\n");
    printf("6. 输出线性表\n");
    printf("0. 退出程序\n");
    printf("===== \n");
    printf("请输入操作编号: ");
}

int main(void) {
    LinkList L;
    int choice;
    int pos;
    char value[50];
    char deleted[50];
    InitList(&L);
    CreateList(L);
    do {
        Menu();
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                if (IsEmpty(L)) {
                    printf("结果: 该线性表为空。 \n");
                } else {
                    printf("结果: 该线性表不为空。 \n");
                }
                break;
            case 2:
                printf("当前线性表长度为: %d\n", GetLength(L));
                break;
            case 3:
                printf("请输入要查询的位置 i: ");
                scanf("%d", &pos);
                if (GetElem(L, pos, value)) {
                    printf("查询结果: 第 %d 个元素是 %s。 \n", pos, value);
                } else {
                    printf("查询失败: 输入的位置不合法。 \n");
                }
            }
        }
    }
}
    
```

```

        break;
    case 4:
        printf("请输入插入位置 i: ");
        scanf("%d", &pos);
        printf("请输入新元素的值: ");
        scanf("%49s", value);
        if (InsertElem(L, pos, value)) {
            printf("插入完成。\\n");
        } else {
            printf("插入失败: 输入的位置不合法。\\n");
        }
        break;
    case 5:
        printf("请输入删除位置 i: ");
        scanf("%d", &pos);
        if (DeleteElem(L, pos, deleted)) {
            printf("删除完成, 被删除的元素为: %s。\\n", deleted);
        } else {
            printf("删除失败: 输入的位置不合法。\\n");
        }
        break;
    case 6:
        PrintList(L);
        break;
    case 0:
        printf("已退出 ADT 表操作程序。\\n");
        break;
    default:
        printf("输入错误, 请重新选择。\\n");
        break;
    }
} while (choice != 0);
DestroyList(&L);
return 0;
}
    
```

六、实验结果

判断线性表是否为空+求线性表长度

```

问题  输出  调试控制台  终端  端口

○ PS D:\Epan\Study\数据结构\23通信\源程序> cd "d:\Epan\Study\数据结构
验1 }
请输入需要建立的元素个数: 4
请输入第 1 个元素的值: 山
请输入第 2 个元素的值: 东
请输入第 3 个元素的值: 大
请输入第 4 个元素的值: 学
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号: 1
结果: 该线性表不为空。
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号: 2
当前线性表长度为: 4
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号: █

```

操作 3，取第二个元素，显示为“东”，通过操作 4 在第三个位置插入新元素 SDU

```

0. 退出程序
=====
请输入操作编号: 3
请输入要查询的位置 i: 2
查询结果: 第 2 个元素是 东。
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号: 4
请输入插入位置 i: 3
请输入新元素的值: SDU
插入完成。
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号: █
    
```

通过操作 3 验证，SDU 元素插入成功，使用操作 5 删除第一个元素“山”，再通

过操作 3 查第一个元素为“东”，证明删除成功

```

=====
请输入操作编号：3
请输入要查询的位置 i：3
查询结果：第 3 个元素是 SDU。
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号：5
请输入删除位置 i：1
删除完成，被删除的元素为：山。
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号：3
请输入要查询的位置 i：1
查询结果：第 1 个元素是 东。
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号：

```

输出线性表并退出程序

```

1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号：6

当前线性表内容为：
第 1 个元素：东
第 2 个元素：SDU
第 3 个元素：大
第 4 个元素：学
1. 判断线性表是否为空
2. 求线性表长度
3. 取第 i 个元素
4. 在第 i 个位置插入元素
5. 删除第 i 个元素
6. 输出线性表
0. 退出程序
=====
请输入操作编号：0
已退出 ADT 表操作程序。
    
```

七、实验心得

通过本次实验，我完成了 ADT 表的基本结构设计与相关操作的编程实现，包括线性表的初始化、判空、插入、删除、按序号查找以及输出等功能。在实验过程中，我对链表这种动态存储结构有了更加直观和深入的理解，也进一步熟悉了 C 语言中结构体、指针和动态内存分配的使用方法。

链表是一种典型的线性数据结构，它不像数组那样要求元素在内存中连续存放，而是通过指针将各个结点按照逻辑顺序连接起来。每个结点通常由数据域和指针域两部分组成，数据域用于存放元素内容，指针域用于指向下一个结点。正是由于这种存储方式，链表在插入和删除操作中具有较高的灵活性，只需要修改相关结点的指针关系，而不需要像顺序表那样移动大量元素。

在程序实现过程中，我重点体会到了链表操作中指针变化顺序的重要性。例如在插入新结点时，必须先让新结点的指针指向原来的后继结点，再让前驱结点的指针指向新结点。如果顺序错误，就可能导致原有链表断开，造成部分结点丢失。通过对插入函数的调试，我进一步理解了链表结构中“前驱结点—新结点—后继结点”之间的连接关系。

在删除操作中，我也认识到边界条件处理非常重要。删除第一个元素、删除最后一个元素以及删除位置不合法等情况都需要单独判断，否则容易出现空指针访问或程序异常。本次代码中采用带头结点的链表结构，使插入和删除操作更加统一，减少了对首元结点的特殊处理，也提高了程序的稳定性和可读性。此外，本次实验还让我更加重视 C 语言中的动态内存管理。链表结点需要通过 malloc 动态申请空间，在删除结点或程序结束时，也需要及时使用 free 释放内

存，避免造成内存泄漏。同时，在字符串输入和复制过程中，也需要注意数组长度限制，防止输入内容过长导致越界问题。

通过本次实验，我不仅掌握了 ADT 表的基本操作实现方法，也进一步加深了对链表存储特点、指针操作和动态内存管理的理解。相比单纯学习理论知识，通过亲自编写和调试程序，我更加清楚地认识到数据结构实现过程中细节处理的重要性。这次实验提高了我分析问题、定位错误和修改程序的能力，也为后续学习栈、队列、树等更复杂的数据结构打下了基础